Cómo funciona Cantera y para qué sirven sus archivos

Guía técnica y de aprendizaje. Actualizada el 16 de octubre de 2026.

Qué hace el sistema

Cantera organiza un torneo infantil: equipos, jugadores, jornadas, goles e incidencias. Produce nóminas para el árbitro, listas de incidencias y goleadores. Los datos se conservan en MySQL; las fotografías se guardan como archivos.

Un visitante puede consultar el torneo sin cuenta. Un administrador autenticado puede registrar, corregir y eliminar información, respetando sus relaciones. No hay registro público de cuentas administradoras ni roles distintos para integrantes del grupo. Tampoco se gestionan partidos completos, clasificaciones por puntos o sanciones por acumulación de tarjetas.

Recorrido de una solicitud

Al abrir `/jugador`, el navegador envía una petición GET. PHP entra por `public/index.php`, Symfony identifica la ruta y llama a `JugadorController`. El controlador consulta jugadores mediante Doctrine y entrega los resultados a una plantilla Twig. Twig genera HTML que el navegador presenta con CSS y JS.

Al guardar un jugador sucede lo siguiente:

1. El navegador envía POST con los campos, el archivo opcional y el token CSRF.

2. Symfony verifica la sesión y que la cuenta tenga `ROLE_ADMIN`.

3. `JugadorType` convierte los campos a tipos PHP y selecciona el equipo existente.

4. Las validaciones comprueban nombres, edad, fecha, equipo y fotografía.

5. Si hay errores, se muestra el formulario con los mensajes; no se guarda.

6. Si hay foto válida, `FotografiaUploader` crea un nombre aleatorio y mueve el archivo.

7. Doctrine escribe la fila con `persist()` y `flush()`; al editar, actualiza la existente.

8. La respuesta redirige a la lista mediante HTTP 303 para evitar repetir el POST al recargar. Los suscriptores agregan cabeceras y registran el resultado de la solicitud.

La base de datos y el archivo de fotografía no forman una transacción conjunta. Si la foto se guarda pero MySQL falla, puede quedar un archivo sin referencia. Las fotos reemplazadas se conservan; no existe una limpieza automática de ellas.

Organización de los datos

Entidad / tabla	Qué contiene	Relaciones
`Equipo` / `equipo`	ID y nombre del equipo.	Un equipo tiene varios jugadores.
`Jugador` / `jugador`	Nombres, apellidos, nacimiento y referencia de fotografía.	Pertenece a un equipo; puede tener goles e incidencias.
`Jornada` / `jornada`	Nombre y fecha de juego.	Puede tener registros de goles.
`Gol` / `gol`	Cantidad total de goles.	Se relaciona con un jugador y una jornada; la combinación es única.
`Incidencia` / `incidencia`	Descripción, tarjeta, fecha y suspensión opcional.	Pertenece a un jugador.
`Administrador` / `administrador`	Usuario único y hash de contraseña.	Otorga el rol de administración de la aplicación.

Doctrine añade una tabla para registrar migraciones aplicadas. Puede existir también la tabla de transporte de Messenger. Un usuario de MySQL, como `cantera`, no es una cuenta de la página: permite a PHP conectar con la base. La cuenta `admin` pertenece a la tabla `administrador` y sirve para iniciar sesión en la web.

Reglas que conviene explicar durante la presentación

- Nombres y apellidos obligatorios: hasta 100 caracteres, letras, tildes, ñ, espacios, guiones y apóstrofes. Se rechazan números y otros símbolos.

- Jugadores de al menos 6 años cumplidos al crear o editar. No hay máximo de edad.

- Fechas reales: no se acepta 30 de febrero. El calendario se comprueba en el servidor.

- El equipo debe existir. La fotografía es opcional, JPEG/PNG/WebP, hasta 2 MB y 4096 × 4096 píxeles.

- Goles: entero de 1 a 100. Un jugador tiene un solo total por jornada.

- Roja: necesita fecha de suspensión; no puede ser anterior a la incidencia. Amarilla: no admite suspensión. La suspensión corresponde a la fecha exacta.

- No se elimina un equipo con jugadores, ni un jugador con goles o incidencias, ni una jornada con goles. Primero deben resolverse las dependencias.

Las reglas del formulario no se ejecutan al cambiar datos directamente con SQL. MySQL sí mantiene sus claves foráneas, campos obligatorios e índice único. Para revisar registros antiguos sin alterarlos se usa `app:auditar-datos`.

Mapa de carpetas

Carpeta	Función	¿Se puede borrar?
`backend/src/`	Lógica PHP: entidades, controladores, formularios, servicios y comandos.	No: es el código del sistema.
`frontend/templates/`	Plantillas Twig que generan las pantallas.	No.
`frontend/assets/`	CSS, JavaScript e imágenes de interfaz.	No. `frontend/assets/vendor` se reinstala con importmap si es necesario.
`public/`	Única raíz que debe exponer el servidor web. Incluye la entrada PHP y assets compilados.	No; los assets compilados pueden regenerarse.
`config/`	Rutas, servicios, seguridad y paquetes Symfony.	No.
`migrations/`	Historial de cambios del esquema. Las activas están en `migrations/mysql`.	No; las antiguas SQLite son historial, no migraciones activas.
`tests/`	Pruebas funcionales y de integración.	Conservar para detectar regresiones.
`resources/demo/`	Datos SQL y avatares ficticios para una instalación nueva.	Conservar si se necesita la demostración.
`vendor/`	Librerías instaladas por Composer.	No mientras se usa la aplicación; se puede reconstruir con `composer install`.
`var/`	Caché, logs, fotos, MySQL local, respaldos y otros datos privados.	Nunca borrar toda la carpeta.
`docs/`	Manual, informes, capturas, guías y PDFs.	Conservar los entregables.
`entrega/`	ZIPs preparados para compartir.	Usar el actualizado; los anteriores pueden estar desfasados.

Archivos importantes de PHP

Archivo	Responsabilidad

`public/index.php`	Crea el Kernel con el entorno y estado de depuración.
`backend/src/Kernel.php`	Arranca Symfony y fija la zona horaria America/Guatemala.
`backend/src/Controller/InicioController.php`	Reúne estadísticas y próximos eventos para el inicio.
`backend/src/Controller/EquipoController.php`	Consulta y gestiona equipos; protege su eliminación.
`backend/src/Controller/JugadorController.php`	Lista, ficha, alta, edición, eliminación y entrega segura de fotos.
`backend/src/Controller/RegistroController.php`	Comparte el flujo de jornadas, goles e incidencias mediante una lista fija de módulos.
`backend/src/Controller/ReporteController.php`	Construye filtros y valida que un jugador corresponda al equipo elegido.
`backend/src/Controller/SecurityController.php`	Presenta login; Symfony intercepta autenticación y logout.
`backend/src/Entity/*.php`	Define campos, relaciones Doctrine y validaciones de negocio.
`backend/src/Form/*Type.php`	Define campos, etiquetas, tipos y restricciones de archivos.
`backend/src/Form/Extension/StrictDateExtension.php`	Valida fechas de calendario para todos los campos HTML de fecha.
`backend/src/Service/EstadisticasTorneo.php`	Suma goles, consulta incidencias, prepara nóminas y estadísticas.
`backend/src/Service/FotografiaUploader.php`	Genera nombres aleatorios, guarda archivos y restringe las rutas de lectura.
`backend/src/EventSubscriber/SecurityHeadersSubscriber.php`	Añade protección HTTP y evita almacenar respuestas dinámicas en cachés.
`backend/src/EventSubscriber/AuditSubscriber.php`	Escribe la bitácora de solicitudes de modificación, sin cuerpos ni secretos.
`backend/src/Repository/*.php`	Punto de acceso a consultas de equipos y jugadores mediante Doctrine.

Configuración y dependencias

Archivo	Para qué sirve
`composer.json`	Declara las librerías y versiones compatibles, PHP requerido y comandos de instalación.
`composer.lock`	Fija las versiones exactas para instalaciones reproducibles. No editar a mano.
`symfony.lock`	Registra recetas/configuraciones aplicadas por Symfony Flex.
`config/services.yaml`	Registra servicios automáticamente y configura el directorio privado de fotografías.
`config/packages/security.yaml`	Hashes, proveedor de usuarios, login, logout, limitador y permisos de rutas.
`config/packages/framework.yaml`	Idioma, secreto, sesión, cookies y proxy de confianza.
`config/packages/doctrine.yaml`	Conexión DATABASE_URL, mapeo y cachés de Doctrine.
`config/packages/doctrine_migrations.yaml`	Selecciona las migraciones MySQL que pueden ejecutarse.
`config/packages/csrf.yaml`	Configura protección CSRF para formularios y autenticación.
`config/packages/monolog.yaml`	Configura errores y bitácora de auditoría.
`config/packages/asset_mapper.yaml`	Expone y versiona assets a través de AssetMapper.
`config/routes.yaml`	Carga las rutas declaradas con atributos en controladores.
`config/reference.php`	Referencia generada de opciones de configuración; no es el lugar para cambiar reglas.
`importmap.php`	Declara los módulos JavaScript usados sin necesitar un compilador Node para servirlos.

`phpunit.dist.xml`	Configura la ejecución de las pruebas y el entorno test.

Docker no se utiliza. Se retiraron los archivos Compose de MySQL/Mailpit que habían quedado como alternativa de desarrollo. Esta copia usa PHP y MySQL instalados en Windows. Frontend y backend siguen siendo una sola aplicación Symfony; no requieren servidores independientes.

Entornos y archivos privados

`.env` contiene valores base; `.env.local` los adapta a esta computadora y no se comparte. Son los únicos dos archivos de entorno en la raíz. La aplicación compartida usa `APP_ENV=prod` y `APP_DEBUG=0`. Los tests fuerzan el entorno `test` mediante `phpunit.dist.xml`; las credenciales opcionales de la base de pruebas están en `tests/.env.local`, excluido de Git y del ZIP.

`APP_SECRET` es un secreto de la aplicación: no es la contraseña de admin ni de MySQL. `DATABASE_URL` reúne motor, usuario, clave, host, puerto y base de datos. La instalación nueva se explica en `docs/Manual-Cantera.pdf`; requiere generar un secreto nuevo. No sobrescribas el `.env.local` de una instalación existente.

La carpeta auxiliar `.codex-tools/` se apartó de la raíz después de las verificaciones y la generación de documentos. Su borrado fue bloqueado por la política de ejecución; permanece archivada en `var/backups/herramientas-retiradas`, fuera de la entrega. La aplicación no depende de Node ni Playwright. Los PDF vigentes se conservan en `docs/`; las fuentes editables están en `var/backups/documentacion-fuentes/`. El manual HTML archivado puede abrirse en el navegador y guardarse como PDF desde Imprimir.

En esta computadora MySQL usa `var/mysql/my.ini` y guarda archivos internos en `var/mysql/data`. No abrir ni alterar esos archivos como si fueran documentos. Para consultar la base se utiliza Workbench, host 127.0.0.1, puerto 3307, esquema `cantera_torneo` y usuario `cantera`. La contraseña se consulta privadamente en `.env.local`. Para respaldar una instancia activa se usa un volcado SQL, no la copia en caliente de archivos internos de InnoDB.

`var/uploads/jugadores` contiene las fotografías; `var/backups` conserva respaldos; `var/torneo.db` es la copia histórica SQLite y no la base activa. `var/cache` es regenerable; `var/log` contiene diagnóstico y auditoría. Limpiar la caché no debe eliminar fotos, registros MySQL ni respaldos.

Comandos útiles

```
# Iniciar MySQL local si aún no está activo
powershell -ExecutionPolicy Bypass -File .\bin\iniciar-mysql.ps1

# Iniciar la página local
php -S 127.0.0.1:8000 -t public bin/router.php

# En otra terminal, compartir temporalmente
& "C:\Program Files (x86)\cloudflared\cloudflared.exe" tunnel --url http://localhost:8000

# Comprobar reglas de los datos sin modificarlos
php bin/console app:auditar-datos --env=prod --no-debug

# Comprobar el código y la base
php bin/phpunit
php bin/console doctrine:schema:validate --env=prod --no-debug
composer audit --locked

# Después de modificar código o estilos en modo producción
php bin/console cache:clear --env=prod --no-debug
php bin/console asset-map:compile --env=prod --no-debug

# Crear una cuenta o cambiar su contraseña con la política habitual
php bin/console app:crear-admin nombre_del_integrante
```

`bin/console` ejecuta comandos Symfony; `bin/phpunit` ejecuta las pruebas; `bin/router.php` adapta el servidor integrado PHP a las rutas de la aplicación; `bin/mysql-console.php` muestra los datos de conexión sin imprimir la contraseña.

`app:demo` crea registros ficticios en una instalación preparada; no debe usarse como reparación de una base existente. `app:import-sqlite` transfiere un respaldo SQLite a un destino vacío, verifica la copia y se niega a sobrescribir registros. Las migraciones se ejecutan al instalar o modificar el esquema, no en cada arranque.

Qué cambiar para una necesidad concreta

Para modificar la edad mínima, revisar `Jugador.php`, el atributo máximo de fecha en `JugadorType.php` y sus pruebas. Para cambiar colores, editar CSS en `frontend/assets/styles`, compilar assets y recargar el navegador. Para un nuevo reporte, añadir la consulta en `EstadisticasTorneo`, filtros/ruta en `ReporteController` y presentación en `frontend/templates/reporte`.

Para añadir una columna, cambiar la entidad y crear una migración revisada; no usar actualizaciones destructivas del esquema en producción. Para cambiar permisos, revisar tanto `security.yaml` como los atributos de controladores y añadir pruebas que intenten acceder sin autorización.

Cómo interpretar las pruebas

`AccesoTest` cubre visitantes, login/logout, CSRF, intentos repetidos y cabeceras. `JugadorFotografiaTest` cubre altas/ediciones inválidas, archivos y conservación de fotos. `TorneoTest` cubre módulos, reportes, fechas, tokens y salida HTML. `MySqlTest` verifica relaciones, Unicode, duplicados y resultados con MySQL real.

Pasar las pruebas confirma esos casos, no todos los posibles. Los pendientes y límites de la revisión están en `informe-auditoria.pdf`.